

Report Analisi codice Python

Consegna S2L5

1. Obiettivo

L'esercizio prevede di allenarci ad avere un'osservazione critica, quindi saper leggere il codice, individuare nello stesso le casistiche non standard che il programma non gestisce, individuare eventuali errori di sintassi ed errori logici; infine proporre una soluzione per ogni errore trovato.

2. Comprensione del codice

Quello che dovrebbe fare il seguente codice è un'*Assistente Virtuale* che, prendendo come input dall'utente una cosa vuoi sapere tra le seguenti richieste:

- Qual è la data di oggi
- La data di oggi
- Che ore sono

che, a seconda della richiesta, l'*Assistente Virtuale* darà le seguenti risposte:

- La data di oggi
- E l'ora attuale

Nel caso l'utente volesse chiedere come si chiama l'utente chiedesse il nome all'*Assistente Virtuale*, l'assistente dirà all'utente che si chiamerà *Assistente Virtuale*, invece nel caso in cui volesse uscire, digitando "esci" potrà uscire dallo stesso

3. Individuazione degli errori sintattici e logici e proposta soluzione

Cominciando a leggere il codice sorgente gli errori sintattici e logici che possiamo notare sono:

- il primo in assoluto è la mancanza dei due punti (:) nella 3^a riga, l'inizio del ciclo while True, questo errore si collega subito al secondo errore
- il secondo errore è che il while True si trova prima del blocco di codice della funzione assistente virtuale (**def assistente_virtuale**)
- il terzo errore che ho individuato si trova alla 13^a riga, dove su **datetime.datetoday()** manca il punto su datetoday(), quindi in realtà la forma corretta sarebbe **datetime.date.today()**
- il quarto errore che ho individuato si trova nella 16^a riga, dove in **datetime.datetime.now().today()** il today non doveva essere scritto quindi la forma corretta sarebbe dovuta essere **datetime.datetime.now()**
- Per il quinto errore dobbiamo tornare all'interno all'interno del ciclo while, nella 7^a riga, dove troviamo un'errore logico, per la precisione di indentazione, dove **break** si dovrebbe trovare sotto il print della 6^a riga

```

1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
23

```

4. Proposta di migliorie

Oltre alla correzione degli errori, descritti nel 3° paragrafo, ho pensato a diverse migliorie, che aiuterebbero il programma a essere più ottimizzato e robusto

La prima miglioria che si potrebbe implementare sarebbe togliere le limitazioni in input da parte dell'utente, aggiungendo prima di ogni confronto **"if"** **.strip().lower()**, grazie all'implementazione di questa miglioria, quando l'utente andrà a "domandare" all'assistente che ore sia o che giorno sia non dovrà obbligatoriamente scrivere "Qual è la data di oggi?", togliendo il problema dell'input *case sensitive* (questa implementazione la implementerei specificatamente nelle righe 5, 12, 15, 18).

La seconda miglioria che andrei ad implementare, per facilitare l'utente ad utilizzare il programma, guidandolo con le scelte che ti propone lo stesso programma, sarebbe quello di aggiungere un menù interattivo. Ciò implicherebbe aggiungere una funzione chiamata **"menu_interattivo()"** che andrà a stampare inizialmente il messaggio iniziale che potrebbe essere "Cosa vuoi sapere" e poi avendo all'interno i comandi disponibili e come uscire dallo stesso.